ESPECIFICACIÓN MAESTRA DEFINITIVA • STACK 100% CLAUDE PRO & GEMINI AI PRO

IURIS-IA / LEX-AGENT ES: ASISTENTE LEGAL AUTÓNOMO

Sistema Agéntico Experto en Derecho Español con Enrutamiento Inteligente sobre Suscripciones Pro (Gemini 3.8 Flash, 4 Argon, Claude Haiku, Sonnet y Opus 5.5 con Niveles de Razonamiento Low/Med/High)

Ámbito Territorial:	España (BOE Consolidado, 17 Regímenes Forales Autonómicos y 30 Convenios Colectivos Clave)
Fuentes Primarias:	BOE Datos Abiertos (REST API), CENDOJ/CGPJ (LanceDB Vectorial + Playwright Stealth) y REGCON (Convenios)
Stack de Modelos:	Google AI Pro / Antigravity: Gemini 3.8 Flash & 4 Argon | Claude Pro: Haiku, Sonnet & Opus 5.5 (Low/Med/High)
Coste Operativo:	0,00 € adicionales en tokens (Aprovechamiento íntegro de las suscripciones planas ya pagadas)
Blindaje Semántico:	Token Fuzzy Matching (Levenshtein ≥ 95%) + Micro-Validador NLI (Natural Language Inference)
Motor Procesal:	Cómputo determinista de días hábiles con 8.131 festivos municipales + Día de Gracia (Art. 135.5 LEC)
Cumplimiento Legal:	Exención Doméstica AI Act (Art. 2.10) + Cortafuegos Anti-Intrusismo (Art. 403 CP y Arts. 23/31 LEC >2.000€)
Testing & Calidad:	Benchmark Ciego con Sentencias Purgadas del Tribunal Supremo y Audiencias Provinciales (Score 0-100)
Pila de Software:	Python 3.11+, FastAPI (Backend SSE), Web UI (Tailwind/React), LanceDB, SQLite3 (WAL), RapidFuzz
Versión del Documento:	Versión 2.1 Maestra Definitiva (Optimizada para el Stack Pro del Usuario)

DIRECTIVA MAESTRA DE INGENIERÍA

DISEÑO A MEDIDA DE TUS SUSCRIPCIONES: Esta versión descarta explícitamente cualquier requisito de hardware local (Ollama/Qwen). La arquitectura está optimizada al 100% para combinar de forma sinérgica tus suscripciones activas de Google AI Pro (Gemini 3.8 Flash y futuro Argon 4) y Claude Pro (Haiku, Sonnet y Opus 5.5 con thinking configurable). El sistema absorbe la lectura masiva gratis con Gemini y reserva el razonamiento profundo de Opus/Argon para la estrategia final, protegiendo tus límites de cuota.

1. Resumen Ejecutivo y Planteamiento del Problema

1.1. El Problema: Asimetría y Brecha en el Acceso a la Justicia

El acceso al asesoramiento jurídico en España presenta una barrera económica y técnica infranqueable para la ciudadanía. Ante contingencias cotidianas —retención indebida de fianzas de alquiler, despidos disciplinarios, discrepancias contractuales bancarias o sanciones administrativas—, el ciudadano común no sabe si su pretensión es legítima ni conoce los plazos procesales perentorios para accionar. El proyecto Iuris-IA / Lex-Agent ES crea un asistente de escritorio autónomo que democratiza el acceso a la información jurídica rigurosa.

1.2. La Tesis del Informático: Orquestación Determinista y Modelos Frontera

El desarrollador del proyecto no necesita ser jurista: la solidez del sistema se logra mediante ingeniería de software determinista (guardarraíles de citación tokenizada, bases vectoriales locales, filtros de congruencia lógica NLI y cómputo matemático de días hábiles) combinada con el razonamiento jurídico superior de los modelos frontera de Anthropic (Opus 5.5 / Sonnet) y Google (Gemini 3.8 Flash / 4 Argon en Antigravity).

2. Análisis Comparativo de Enfoques de Inteligencia Artificial

Enfoque Técnico	Mecanismo Operativo	Comportamiento en Derecho Español	Veredicto Técnico
1. Fine-Tuning (LoRA / QLoRA)	Reentrenamiento de pesos sinápticos con leyes y demandas.	Aprende estilo procesal forense, pero comprime la información en pesos estáticos.	Inviable en solitario: Alucina números de artículos y cita normas derogadas al carecer de actualización dinámica.
2. Inyección Masiva (Long Context Dump)	Volcar códigos enteros en ventanas de 1M de tokens.	El corpus español supera los 120M tokens (10.000+ normas BOE + CC.AA.). Sufre Lost in the Middle.	Inviable en escala: Ceguera ante la jerarquía normativa y excepciones transitorias; latencia y costes inasumibles.
3. Agentic RAG Quirúrgico + MCP + NLI	Recuperación atómica de 10-15 artículos y 3-5 sentencias en ventana limpia con verificación lógica.	Máxima precisión factual, trazabilidad oficial completa y verificación determinista en Python.	Estándar Óptimo: Fiabilidad matemática, ausencia de alucinaciones y coste por consulta de 0,00 € sobre tus planes Pro.

3. Fuentes de Datos e Ingeniería de Acceso Desacoplada

Para garantizar una respuesta instantánea sin bloqueos en el portal del CGPJ y resolver el vacío de los convenios laborales, el sistema adopta una **Capa de Datos Híbrida Desacoplada**:

Fuente Jurídica	Cobertura y Contenido	Mecanismo de Acceso en Iuris-IA
BOE Legislación Consolidada	Leyes estatales consolidadas vigentes, códigos básicos, reformas y enlaces de análisis.	API REST oficial pública (GET) a /datosabiertos/api/legislacion-consolidada con parser Pydantic polimórfico.
CENDOJ (CGPJ) Jurisprudencia	Millones de sentencias de Tribunal Supremo, Audiencia Nacional, TSJ y Audiencias Provinciales.	Cache-First en LanceDB local (5.000 precedentes clave a <50ms) + fallback asíncrono con playwright-stealth.
Convenios Colectivos (REGCON / BOP)	Convenios sectoriales y de empresa (salarios reales, categorías, pluses, faltas laborales).	Pre-indexación local de los 30 convenios más masivos de España + selector UI para carga de PDF/nómina.
Ordenanzas Municipales (BOP)	Normativa local: multas ORA/SER, ruidos, licencias y Plusvalía municipal.	Gatekeeper de competencia local en Triaje; solicitud de PDF de la sanción al usuario y advertencia de primacía local.

3.1. Pipeline de Extracción Robusta de Sentencias (OCR y Ligaduras)

connectors/pdf_extractor.py aplica una triple salvaguarda: 1) Conmutación a OCR local (Tesseract / Windows OCR) si pdfplumber extrae menos de 100 caracteres por página (PDF escaneado); 2) Normalización tipográfica de ligaduras Unicode (\ufb01 → fi, \ufb02 → fl) y des-silabeo con regex; y 3) Aislamiento determinista de VOTOS PARTICULARES disidentes para impedir que se adopten como doctrina de la Sala.

4. Sincronización Temporal y Validador de Reformas Normativas

El módulo core/temporal_sync.py previene el anacronismo jurisprudencial cruzando la fecha_resolucion de la sentencia con la fecha_actualizacion y referencias_posteriores del BOE. Si el precepto sufrió reformas con posterioridad al fallo, inyecta una advertencia imperativa de desincronización, verificando asimismo si la controversia se rige por disposiciones transitorias o por el principio de irretroactividad de las normas desfavorables (Art. 9.3 y 25 CE).

5. Arquitectura del Pipeline Agéntico en 4 Fases

El orquestador divide la ejecución en cuatro etapas desacopladas con **Streaming Progresivo SSE en la Web UI** y asignación óptima de tus modelos suscritos:

Fase y Latencia	Misión y Asignación de Modelos	Entrada / Salida y Optimización
Fase 1: Triaje Multihop (1 - 2 seg)	Traductor Jurídico Invisible. Motor: Claude Haiku o Gemini 3.8 Flash (Low effort).	In: Texto usuario. Out: JSON con leyes BOE y términos CENDOJ. Lanza preguntas aclaratorias si hay ambigüedad.
Fase 2: Cribado Pesado (2 - 4 seg)	Cribado de BOE, convenios y PDFs CENDOJ. Motor: Gemini 3.8 Flash (AI Studio Free Tier / Pro).	In: JSON Fase 1. Out: Dossier compacto (~3.000 tokens). Absorbe gratis miles de tokens de PDFs judiciales.
Fase 3: Debate Dialéctico (10 - 25 seg stream)	Contradictorio procesal: Defensa vs Contraparte vs Magistrado. Motor: Claude Opus 5.5 / Sonnet o Gemini 4 Argon.	In: Dossier curado de Fase 2. Out: Dictamen bajo lógica disyuntiva estricta con niveles de thinking (Low/Med/High).
Fase 4: Blindaje NLI (1 - 3 seg)	Guardarraíles en Python: Fuzzy Matching (≥95%), NLI y calendario judicial. Motor: Python local + Gemini Flash.	In: Dictamen Fase 3. Out: Dictamen certificado con advertencia deontológica de postulación legal.

6. Guardarraíles Deterministas y Blindaje Semántico NLI

- Token Fuzzy Matching (Levenshtein ≥ 95%): El dictamen exige el campo verbatim_quote. El validador en Python normaliza comillas, espacios y signos de puntuación, calculando la similitud tokenizada frente al texto descargado del BOE. Si el ratio es inferior al 95%, bloquea la emisión.
- Micro-Validador de Congruencia Lógica NLI: Mediante un prompt estructurado de entailment con Gemini 3.8 Flash, se comprueba que la conclusión del asistente no contradiga la literalidad del precepto legal invocado.
- Motor de Calendario Judicial y Días Hábiles (calendar_engine.py): Integra el dataset oficial de festivos locales de los **8.131 municipios de España** más festivos autonómicos y nacionales, aplicando estrictamente la regla del Día de Gracia Procesal (Art. 135.5 LEC: plazo hábil hasta las 15:00 horas del siguiente día hábil).
- Tratamiento de Conceptos Jurídicos Indeterminados (CJI): Cuando se invocan estándares elásticos (*'pequeñas reparaciones'*, *'buena fe'*), el sistema prohíbe inventar importes con base en el BOE y obliga a fundamentar la cuantía exclusivamente en sentencias de Audiencias Provinciales.

7. Matriz de Enrutamiento Inteligente sobre tus Suscripciones

Modo / Perfil	Fase 1 (Triaje) & Fase 2 (PDFs)	Fase 3 (Dictamen Senior)	Impacto en Cuota / Latencia
1. Modo Eco (Día a Día)	Gemini 3.8 Flash (Low effort / AI Studio Free)	Gemini 3.8 Flash (High effort) o Claude Haiku	0% cuota Claude Pro. Latencia: ~6 seg.
2. Equilibrado (Recomendado)	Gemini 3.8 Flash (Absorbe lectura masiva)	Claude Sonnet 3.5 / 4.5 (Medium effort)	~3% cuota Claude Pro. Latencia: ~14 seg.
3. Modo Pro (Tribunal Supremo)	Gemini 3.8 Flash (Dossier curado a 3.000 tok)	Claude Opus 5.5 o Gemini 4 Argon (High effort / deep reasoning)	~9% cuota Claude Pro. Latencia: ~28 seg stream.
4. Auditoría Cruzada (Consenso Dual)	Gemini 3.8 Flash (Lectura y síntesis)	Claude Opus 5.5 (Dictamen) + Gemini 4 Argon (Auditor)	Máximo rigor procesal. Latencia: ~35 seg.

8. Cumplimiento Regulatorio, Deontología y AI Act

- Encuadre Legal bajo el AI Act (Reglamento UE 2024/1689): El sistema se define estrictamente como una Herramienta Personal de Investigación Documental y Soporte a la Auto-Defensa Ciudadana, quedando amparada en la exención de actividad privada y doméstica (Art. 2.10) cuando es ejecutada en local por el propio usuario.
- Erradicación del Delito de Intrusismo (Art. 403 CP): Se prohíben de forma absoluta las predicciones numéricas pseudo-científicas de victoria (*'viabilidad: 78%'*). Se sustituyen por bandas cualitativas de solidez argumental (*'Criterio consolidado favorable'*, *'Disparidad de criterio'*, *'Riesgo probatorio alto'*).
- Alerta Preceptiva de Postulación (Arts. 23 y 31 LEC): Cuando el litigio supere los 2.000 € o se trate de materias con postulación obligatoria (juicio ordinario civil, recurso contencioso o demanda judicial de despido), el sistema bloquea la generación de escritos formales y emite un requerimiento no saltable de contratación de Abogado y Procurador colegiados.

9. Motor de Backtesting Ciego Certificado (Blind Judge)

El motor benchmark/blind_judge.py evalúa el sistema frente a 25 casos reales (15 del Tribunal Supremo y 10 de Audiencias Provinciales) aplicando un estricto **Protocolo Anti-Fuga de Datos**:

1. Purga Forense de Antecedentes: Eliminación de toda mención a las resoluciones de primera instancia y apelación contenidas en los Antecedentes de Hecho, dejando solo la narración fáctica pura.
2. Casos Sintéticos con Permutaciones Controladas: Modificación de fechas, nombres de empresas y cuantías preservando la ratio decidendi para medir deducción jurídica genuina sin atajos memorísticos.
3. Rúbrica de Puntuación (0-100 Puntos): Outcome Match (30 pts), Statutory Recall BOE (30 pts), Alineación Doctrinal con Audiencias y Supremo (25 pts) y Tasa Cero de Alucinación Normativa (15 pts).

10. Estructura del Software y Esquema de Base de Datos

```
abogado_ia_app/
├── config/
│   ├── profiles.json          # Modos: Eco (Flash), Equilibrado (Sonnet) y Pro (Opus/Argon)
│   └── settings.py            # Rutas LanceDB, SQLite WAL, timeouts y effort levels (Low/Med/High)
├── core/
│   ├── router.py              # Adaptador unificado: Google GenAI SDK (Flash/Argon) + Claude CLI
│   ├── orchestrator.py        # Pipeline 4 fases con streaming SSE y WebSocket
│   ├── validators.py          # Token Fuzzy Match (≥95%) y Micro-validador NLI
│   └── calendar_engine.py     # Motor 8.131 festivos municipales + Día de Gracia (Art. 135.5 LEC)
├── connectors/
│   ├── boe_client.py          # Cliente REST oficial /datosabiertos/ con Pydantic polimórfico
│   ├── cendoj_vector.py       # Interfaz LanceDB / SQLite-VSS (5.000 sentencias en local)
│   ├── cendoj_stealth.py      # Worker Playwright-Stealth para búsquedas background
│   ├── convenios_db.py        # Gestor de los 30 convenios colectivos sectoriales masivos
│   └── pdf_extractor.py       # Extractor con OCR de respaldo y aislamiento de Votos Particulares
├── benchmark/
│   ├── blind_judge.py         # Orquestador con purga de antecedentes y casos sintéticos
│   ├── evaluator.py           # Evaluador cuantitativo (Outcome, Recall, Doctrina, Alucinación)
│   └── trap_cases.json        # 25 casos calibrados (TS + Audiencias Provinciales)
├── storage/
│   ├── database.py            # SQLite3 con PRAGMA journal_mode=WAL y busy_timeout=5000
│   ├── local_cache.db         # Tablas: boe_bloques, cendoj_sentencias, convenios, sesiones
│   └── lancedb_store/         # Base vectorial para búsqueda semántica instantánea (<50ms)
└── ui/
    ├── web_server.py          # Servidor FastAPI local con endpoints REST y SSE
    └── web_frontend/          # Interfaz Web moderna (Tailwind, visor de artículos y chat SSE)
```

10.1. Esquema Relacional Consolidado (SQLite3 en Modo WAL)

```
PRAGMA journal_mode=WAL;
PRAGMA busy_timeout=5000;

-- Normativa Consolidada del BOE
CREATE TABLE IF NOT EXISTS boe_bloques (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    id_norma TEXT NOT NULL,                -- BOE-A-1994-26003 (LAU)
    id_bloque TEXT NOT NULL,               -- a21
    titulo TEXT,
    texto_consolidado TEXT NOT NULL,
    fecha_actualizacion TEXT,
    cached_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(id_norma, id_bloque)
);

-- Precedentes Judiciales CENDOJ (TS y Audiencias Provinciales)
CREATE TABLE IF NOT EXISTS cendoj_sentencias (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    roj_id TEXT UNIQUE NOT NULL,           -- STS 4512/2023 o SAP M 1204/2022
    ecli TEXT,
    tribunal TEXT NOT NULL,
    jurisdiccion TEXT NOT NULL,
    fecha_resolucion TEXT NOT NULL,
    antecedentes_purgados TEXT,
    fundamentos_mayoria TEXT,
    voto_particular TEXT,                  -- Aislado de la doctrina vencedora
    fallo TEXT,
    cached_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Convenios Colectivos Sectoriales y Provinciales Masivos
CREATE TABLE IF NOT EXISTS convenios_cache (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    codigo_convenio TEXT UNIQUE NOT NULL,  -- Código oficial REGCON/BOP
    denominacion TEXT NOT NULL,
    ambito_territorial TEXT NOT NULL,      -- Madrid, Barcelona, etc.
    sector_actividad TEXT NOT NULL,        -- Hostelería, Metal, Comercio, etc.
    texto_integro TEXT NOT NULL,
    cached_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Sesiones y Dictámenes Auditados
CREATE TABLE IF NOT EXISTS sesiones_consultas (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    modo_utilizado TEXT NOT NULL,          -- Eco, Equilibrado, Pro
    consulta_usuario TEXT NOT NULL,
    triage_json TEXT NOT NULL,
    articulos_citados TEXT,
    sentencias_citadas TEXT,
    fuzzy_match_score REAL,
    dictamen_markdown TEXT NOT NULL
);
```

11. Hoja de Ruta de Implementación en 5 Sprints

Sprint / Hito	Módulos e Hitos de Ingeniería	Criterio de Éxito / Métrica
Sprint 1: Core & Router Pro	1. Conector Gemini 3.8 Flash / 4 Argon (Google GenAI) + Claude CLI serializado. 2. Inicialización de SQLite WAL y LanceDB local.	Inferencia streaming <10s y conmutación de modo sin colisiones de I/O.
Sprint 2: Datos & Convenios	1. Cliente BOE REST con Pydantic polimórfico. 2. Pre-indexación de 30 convenios masivos y LanceDB con 5.000 sentencias.	Statutory Recall ≥ 92% en recuperación de normas y convenios.
Sprint 3: Calendario & Guardarraíles	1. Motor calendar_engine.py con 8.131 municipios + Día de Gracia. 2. Token Fuzzy Matching (≥95%) y validador de congruencia NLI.	Cero errores en cómputo de plazos procesales. Tasa de alucinación = 0%.

Sprint 4: Pipeline & Backtesting	1. Pipeline 4 fases con debate disyuntivo y streaming progresivo. 2. Ejecución del benchmark ciego con 25 sentencias purgadas.	Doctrinal Alignment ≥ 88% y Outcome Match ≥ 90% en Audiencias y TS.
Sprint 5: Web UI & Certificación	1. Interfaz Web moderna con streaming SSE, visor lateral y soporte PWA. 2. Cortafuegos deontológico y advertencia preceptiva Arts. 23/31 LEC.	Aplicación de escritorio terminada, empaquetable y lista para uso.

11.1. Adaptador Python en core/router.py para tus Modelos

```
import subprocess, json
from google import genai

class ProLegalRouter:
    def __init__(self, gemini_api_key: str):
        self.gemini_client = genai.Client(api_key=gemini_api_key)

    def execute_stage(self, stage: str, profile: str, prompt: str, system_prompt: str, effort: str = "medium") -> str:
        # Enrutamiento inteligente segun fase y perfil seleccionado
        if stage in ["triage", "pdf_screening"]:
            # Fase 1 y 2: Gemini 3.8 Flash (rapido, ventana 1M tokens, coste 0)
            res = self.gemini_client.models.generate_content(
                model="gemini-3.8-flash", contents=f"SYSTEM: {system_prompt}\n\nUSER: {prompt}"
            )
            return res.text

        elif stage == "dialectical_counsel":
            if profile == "eco":
                # Modo Eco: Gemini 3.8 Flash con thinking alto
                res = self.gemini_client.models.generate_content(
                    model="gemini-3.8-flash", contents=f"SYSTEM: {system_prompt}\n\nUSER: {prompt}"
                )
                return res.text
            elif profile == "pro":
                # Modo Pro: Claude Opus 5.5 con thinking profundo via CLI Pro
                cmd = ["claude", "-p", f"{system_prompt}\n\n{prompt}",
                       "--model", "opus", "--effort", effort, "--output-format", "json"]
                res = subprocess.run(cmd, capture_output=True, text=True, check=True)
                return json.loads(res.stdout).get("result", "")
            else:
                # Modo Equilibrado: Claude Sonnet con thinking medio
                cmd = ["claude", "-p", f"{system_prompt}\n\n{prompt}",
                       "--model", "sonnet", "--output-format", "json"]
                res = subprocess.run(cmd, capture_output=True, text=True, check=True)
                return json.loads(res.stdout).get("result", "")
```

11.2. Conclusión y Veredicto Final

Esta Especificación Maestra Definitiva acredita la total viabilidad técnica, operativa y económica de Iuris-IA / Lex-Agent ES utilizando exclusivamente tu ecosistema de suscripciones ya pagadas (Claude Pro y Google AI Pro). Se erradica cualquier dependencia de hardware local costoso y se asegura un rendimiento de nivel profesional sin gastar un solo euro adicional en tokens de API.